

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Backend y Exposición de Servicios para la Base de Datos “Centros de Evaluación de Manejo”

Continuación Práctica 1 y 2

PONDERACIÓN: 35.72 pts

 **Tiempo estimado: 48 hrs/min**

Índice

1. MARCO FORMATIVO	3
1.1. Valor.....	3
1.2. Competencia(s).....	3
1.3. Objetivo SMART.....	3
2. Resumen Ejecutivo	4
3. Enunciado del Proyecto	4
3.1 Descripción del problema a resolver.....	4
3.2 Alcance del proyecto	4
3.3 Entregables	5
4. Material de apoyo	7
5. Metodología.....	7
6. Recursos y herramientas a utilizar	8
7. Desarrollo de Habilidades Blandas.....	8
8. Cronograma.....	9
9. Rúbrica de Calificación.....	10
9.1 Requisitos para optar a la calificación	10
9.2 Resumen de Puntuaciones.....	12
9.5 Comentarios Generales.....	15
Detalle de la Calificación	15

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Integridad Académica	Se fomenta la creación de soluciones originales, prohibiendo copias totales o parciales y promoviendo el desarrollo individual del backend

1.2. Competencia(s)

Tipo de Competencia	
Competencia General	Diseñar y administrar soluciones de bases de datos relacionales para resolver problemas complejos de gestión de información.
Competencia Específica	Implementar arquitecturas de software modernas que integren bases de datos con servicios API REST utilizando contenedores para asegurar la portabilidad y el aislamiento de recursos.

1.3. Objetivo SMART

SMART	Definición	Objetivo redactado
Específico (¿Qué?)	El objetivo es concreto y tangible.	Dockerizar una base de datos Oracle, inicializarla automáticamente con el script DDL y exponer sus datos mediante un API REST en Node.js.
Medible (¿Cuánto?)	El objetivo tiene una medida objetiva de éxito.	El 100% de las operaciones de datos (CRUD y consultas) deben realizarse exitosamente a través de la API, verificadas en Postman.
Alcanzable (¿Cómo?)	El objetivo debe ser posible con los recursos disponibles.	Utilizando Docker Desktop, Node.js/Express, y DBeaver para la gestión y validación de la estructura de la base de datos
Realista (¿Para qué?)	El objetivo contribuye a metas más amplias.	Desarrollar habilidades en backend y DevOps que permitan crear sistemas escalables y desacoplados para el manejo de centros de evaluación.

A Tiempo (¿Cuándo?)	El objetivo tiene fecha límite o mejor aún un cronograma de hitos de progreso	El proyecto debe estar desplegado, documentado en un repositorio de GitHub y listo para calificación en un ciclo de desarrollo de aproximadamente 10 días.
--------------------------------	---	--

2. Resumen Ejecutivo

Este proyecto consiste en la evolución del sistema de gestión para los Centros de Evaluación de Manejo de Guatemala hacia una arquitectura de servicios moderna. El problema por resolver es la transición de una base de datos estática a un ecosistema funcional que garantice portabilidad y seguridad en el acceso a la información. La solución propuesta incluye la dockerización de la base de datos Oracle y el desarrollo de un API REST en Node.js/Express, lo que permitirá centralizar todas las operaciones de datos de forma controlada y reproducible. A través de este proyecto, se pondrán en práctica competencias críticas en contenerización de servicios, desarrollo de backend para la exposición de servicios y validación técnica mediante herramientas de administración de bases de datos y consumo de endpoints.

3. Enunciado del Proyecto

3.1 Descripción del problema a resolver

Actualmente, los Centros de Evaluación de Manejo en Guatemala gestionan la información de aspirantes y exámenes de forma aislada, lo que dificulta la centralización de datos estadísticos y la disponibilidad de la información en tiempo real. Tras haber diseñado el modelo relacional en fases previas, el problema ahora radica en la falta de una infraestructura tecnológica que permita el acceso seguro, portable y estandarizado a estos datos. Se requiere transformar la base de datos estática en un sistema dinámico accesible mediante servicios web, eliminando la manipulación directa de las tablas por parte de los usuarios y asegurando que el entorno de ejecución sea idéntico para cualquier desarrollador o administrador.

3.2 Alcance del proyecto

Alcance obligatorio:

- **Contenerización:** Crear un entorno reproducible utilizando Docker para la base de datos Oracle XE.
- **Automatización:** Configurar la carga automática del esquema DDL al iniciar el contenedor.
- **Capa de Servicios:** Desarrollar un API REST que exponga operaciones CRUD (Crear, Leer, Actualizar, Borrar) para todas las tablas del modelo.
- **Consultas Estadísticas:** Implementar tres endpoints específicos para obtener promedios de punteos, rankings de evaluados y análisis de dificultad de preguntas.
- **Validación:** Documentar y realizar pruebas de todos los endpoints mediante una colección de Postman.

Alcance opcional: Uso de variables de entorno para parametrizar las credenciales de conexión y puertos del servicio.

4.3 Requerimientos técnicos

Los estudiantes deberán integrar las siguientes tecnologías y herramientas para el desarrollo de la solución:

- **Infraestructura:** Docker y Docker Compose para el orquestado de los servicios.
- **Motor de Base de Datos:** Oracle Database (versión XE recomendada).
- **Administración de BD:** DBeaver como cliente para la verificación de estructuras y visualización de datos.
- **Lenguaje de Programación:** Node.js utilizando el framework Express para la construcción del API REST.
- **Versionamiento:** Git y GitHub para el alojamiento del código fuente y documentación.
- **Pruebas de Software:** Postman para el consumo y validación de los servicios expuestos.

3.3 Entregables

Tipo	Descripción
Repositorio de Código	Repositorio privado en GitHub con el nombre SBD1B_1S2026_#carnet . Debe incluir el código fuente del API REST con una estructura clara y el tutor correspondiente agregado como colaborador .
Infraestructura (Docker)	Archivo docker-compose.yml que configure el contenedor de la base de datos Oracle XE , gestione variables de entorno y asegure la persistencia de datos.
Scripts de Inicialización	Archivo SQL con el esquema DDL completo de la Práctica 1, configurado para cargarse automáticamente al levantar el contenedor de la base de datos.

Documentación Técnica (README)	Archivo README.md detallado que incluya: descripción del proyecto, guía paso a paso para el despliegue con Docker, instrucciones de conexión desde DBeaver y capturas de pantalla como evidencia de funcionamiento.
Archivo JSON	Archivo JSON que contenga todas las peticiones a los endpoints de la API (CRUD y Consultas), incluyendo ejemplos de éxito y error para facilitar la calificación.

4. Material de apoyo

- Ejemplos impartidos en clase.
- Instalar Oracle 21c express en un contenedor Docker:
<https://www.youtube.com/watch?v=-UCwWSqcsCo>

5. Metodología

Para el desarrollo exitoso de esta fase, se recomienda seguir un proceso estructurado dividido en las siguientes etapas:

Configuración de Infraestructura y Contenerización:

- Los estudiantes deben investigar y configurar el entorno de Docker y Docker Compose para orquestar los servicios necesarios.
- Se debe establecer el volumen de persistencia y la variable de entorno para la base de datos Oracle XE.
- Configurar la carga automática del script DDL (desarrollado en la Práctica 1) para que el esquema se genere al iniciar el contenedor.

Verificación y Administración de Datos:

- Establecer la conexión desde DBeaver hacia el contenedor de Oracle para validar que las tablas y restricciones se hayan creado correctamente.
- Realizar la carga de datos transaccionales para asegurar que existan registros para las pruebas de las consultas por medio de los datos proporcionados.

Desarrollo del Backend y Exposición de Servicios:

- Implementar el servidor utilizando Node.js y Express, configurando el driver de conexión hacia la base de datos dockerizada.
- Desarrollar los endpoints para las operaciones CRUD de cada tabla del modelo relacional.
- Programar la lógica de las consultas estadísticas solicitadas (promedios, rankings y análisis de preguntas).

Validación y Documentación Técnica:

- Realizar pruebas integrales de cada endpoint utilizando Postman para verificar que las respuestas coincidan con los datos en la base de datos.
- Documentar el proceso de despliegue en el archivo README, detallando cómo cualquier usuario puede levantar el sistema con un solo comando de Docker.
- Exportar la colección de Postman como evidencia final de la funcionalidad del API.

6. Recursos y herramientas a utilizar

Software de Infraestructura y Base de Datos:

- **Docker & Docker Compose:** Para la creación de contenedores y gestión de volúmenes de persistencia.
- **Oracle Database XE:** Motor de base de datos relacional donde se alojará el modelo de los Centros de Evaluación.
- **DBeaver:** Herramienta cliente para la administración de la base de datos, verificación de esquemas y ejecución de scripts de auditoría.

Software de Desarrollo Backend:

- **Node.js & Express:** Entorno de ejecución y framework para la construcción del API REST que expondrá los servicios del sistema.

Plataformas de Gestión y Entrega:

- **GitHub:** Para el control de versiones del código fuente, documentación y colaboración técnica.
- **Postman:** Para el diseño, consumo, pruebas y documentación de los endpoints del API REST.

UEDI / Classroom: Plataformas oficiales para la entrega del enlace del repositorio.

Lecturas y Documentación Recomendada:

- **Documentación oficial de Docker:** Guías sobre docker-compose.yml y manejo de volúmenes.
- **Documentación de Express.js:** Guía para la creación de rutas y controladores de respuesta.
- **Manual de Referencia SQL de Oracle:** Para la optimización de las consultas estadísticas y el manejo de tipos de datos complejos en el DDL.

7. Desarrollo de Habilidades Blandas

Más allá del dominio técnico en bases de datos y desarrollo backend, este proyecto está diseñado para fortalecer competencias transversales esenciales en el perfil de un Ingeniero en Ciencias y Sistemas:

- **Autogestión del Tiempo y Organización:** El estudiante debe planificar de forma independiente la transición entre la infraestructura (Docker), el desarrollo (API) y la documentación, cumpliendo con los hitos de entrega establecidos.
- **Responsabilidad y Ética Profesional:** Se fomenta el compromiso individual con la calidad del código y la integridad académica, asumiendo la responsabilidad total de cada fase del proyecto y evitando prácticas de copia.

8. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	15-04-2026	15-04-2026
Elaboración	15-04-2026	30-04-2026
Calificación	02-05-2026	03-05-2026

9. Rúbrica de Calificación

9.1 Requisitos para optar a la calificación

El incumplimiento de cualquiera de estos puntos puede invalidar la calificación del proyecto 2.

Tema	Descripción	Cumple (Sí/No)
------	-------------	----------------

Infraestructura Obligatoria	La base de datos Oracle debe estar correctamente dockerizada y ser reproducible mediante el archivo docker-compose.yml.	
Integridad de Datos	El sistema debe inicializar el esquema DDL automáticamente al levantar el contenedor, sin intervención manual.	
Capa de Servicios	El repositorio en GitHub debe incluir al tutor como colaborador y contener el historial de commits del estudiante.	
Validación de Endpoints	Se debe presentar evidencia de las pruebas en Postman con los endpoints CRUD y las consultas estadísticas solicitadas.	
Documentación y Evidencia	El archivo README.md debe incluir los pasos de despliegue, guía de DBeaver y capturas de pantalla que evidencien el funcionamiento y todo el proceso implicado.	
Originalidad	El proyecto es estrictamente individual. Cualquier indicio de plagio o copia total/parcial resultará en una nota de 0 puntos.	

9.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Habilidades		
Configuración de Docker y Docker Compose (Levantamiento de BD y persistencia)	10	
Inicialización automática del esquema DDL al iniciar el contenedor	5	
Calidad de código en la API	5	

Implementación de endpoints CRUD para todas las tablas del modelo	10	
Preguntas teóricas	10	
Sub-Total Habilidades	40	
2. Conocimiento		
Consulta 1: Estadísticas de evaluaciones por centro y escuela (Mostrar Total de exámenes, Promedio examen teórico, Promedio examen práctico y Aprobados)	10	
Consulta 2: Ranking de evaluados por resultado final (Ordenamiento y lógica)	10	
Consulta 3: Identificación de la pregunta con menor aciertos (Basarse y poner en la salida porcentaje de aciertos)	10	
Pruebas en Postman	30	
Sub-Total Conocimiento	60	
TOTAL	100	

10. Comentarios Generales

Penalizaciones y consideraciones

Penalizaciones	Porcentaje de la nota total
Si se detecta plagio, copia total/parcial o apoyo de terceros durante la resolución, la nota será de 0 puntos	(-100%)

Si se identifican formatos, estructuras o redacciones sospechosamente similares en la documentación entre distintos estudiantes, se penalizará con un -20% de la nota final.	(-20%)
Cualquier trabajo entregado después de la fecha de entrega será penalizado.	(-100%)
Si al iniciar la calificación el estudiante no tiene el entorno Docker levantado y la conexión a Oracle establecida	(-30%)
El uso de Oracle es obligatorio. El uso de herramientas distintas es motivo de penalización.	(-50%)
Si el estudiante demuestra no manejar la parte práctica o no sabe explicar la lógica de su código/consultas	(-30%)

- Es requisito indispensable contar con **cámara y micrófono** activos durante toda la sesión de calificación virtual. (Verificar su buen funcionamiento)
- El estudiante debe presentarse puntualmente en su horario asignado. De lo contrario, **perderá el derecho a calificación**, salvo que presente una excusa formal con constancia comprobable. No se permiten cambios de horario injustificados o autorizados por los tutores académicos.
- **El estudiante debe apuntarse en los horarios habilitados por su auxiliar correspondiente.** El incumplimiento de esta disposición resultará en la pérdida del derecho a calificación, sin excepción, con el fin de respetar la programación establecida para el resto del grupo.
- Es obligatorio que se agregue al tutor correspondiente al repositorio de Github.
Auxiliar 1: parguet
Auxiliar 2: Tefy1317